

CASE STUDY · UI/UX CAPSTONE

Redesigning a mobile checkout for clarity, trust, and accessibility.

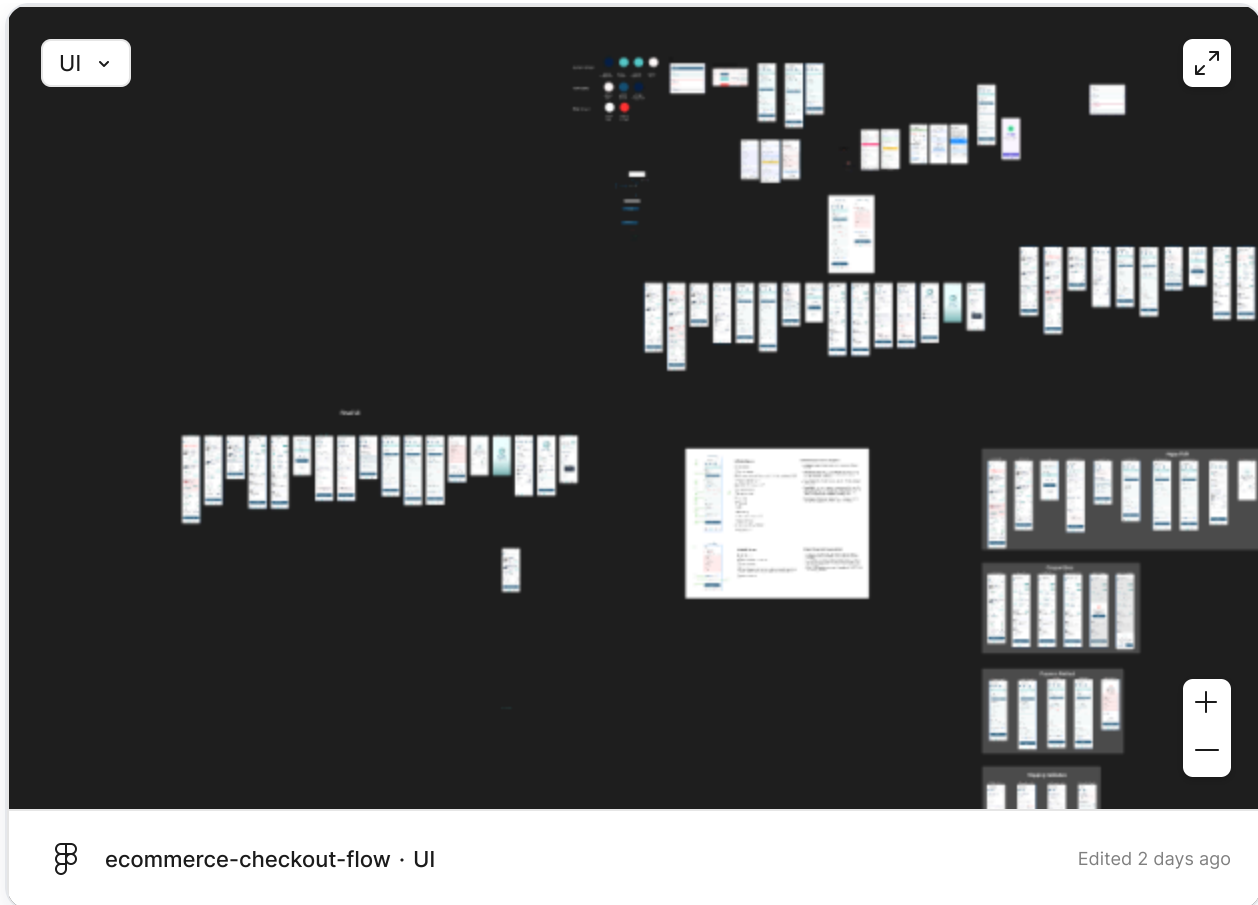
An 8-week capstone project that took a cluttered e-commerce checkout and rebuilt it around a calmer visual system, explicit error recovery, and WCAG 2.2 AA accessibility — covering research, user flows, design tokens, hi-fi UI, prototyping, and testing.

Role: UI/UX designer **Timeline:** 8 weeks **Platform:** Mobile-first

Tools: Figma, Canva, VS Code, Hugo

Interactive prototype

Explore the full checkout flow prototype below — tap through the happy path, coupon, payment, and shipping flows directly in Figma.

**4**

error-recovery
flows designed

19

contrast checks
audited

AA

WCAG 2.2
target met

1

A/B test
proposed

01 Problem

Mobile e-commerce checkouts lose users at the moment that matters most — when they're ready to pay. Common drop-off causes surfaced during the project: unclear pricing (MRP, taxes, shipping, discounts, platform fees), no obvious guest-checkout option, harsh colors causing eye strain, and hard failure states with no recovery path for declined cards, invalid coupons, or shipping errors.

Core insight. Users care about the *final price they pay* and whether they feel they're getting a fair deal before purchasing. Everything else — speed, trust, polish — only matters once that's clear.

02 Users

The checkout was designed for mobile-first shoppers who value speed, transparency, and control — especially first-time buyers without an account and returning users hitting a payment failure.



Priya, 27

First-time mobile shopper

- Doesn't want to create an account
- Needs the total price up front
- Loses trust on unexpected fees



Aman, 34

Returning buyer, card user

- Expects saved details to work
- Gets frustrated by declined cards with no guidance
- Wants quick retry, not a dead end

03 Constraints

The biggest constraints came from the prototyping tool itself: Figma could not simulate real application behaviour, so certain interactions could only be

documented, not demonstrated. Across weeks 5–7 these limitations forced explicit, reviewable design decisions instead of hidden workarounds.

Week 5 · Keyboard nav

No real Tab-order focus traversal exists inside a Figma prototype — focus can't be demonstrated interactively.

→ **Intended focus order documented as numbered tab-order tables (see Testing).**

Week 6 · Live inputs

No live text entry, blinking caret, auto-focus movement, or real-time form validation.

→ **Built explicit component variants for every state (Default → Disabled).**

Week 6 · Branching

A single interaction can't branch to multiple outcomes — Pay Now can't route to success or failure based on input.

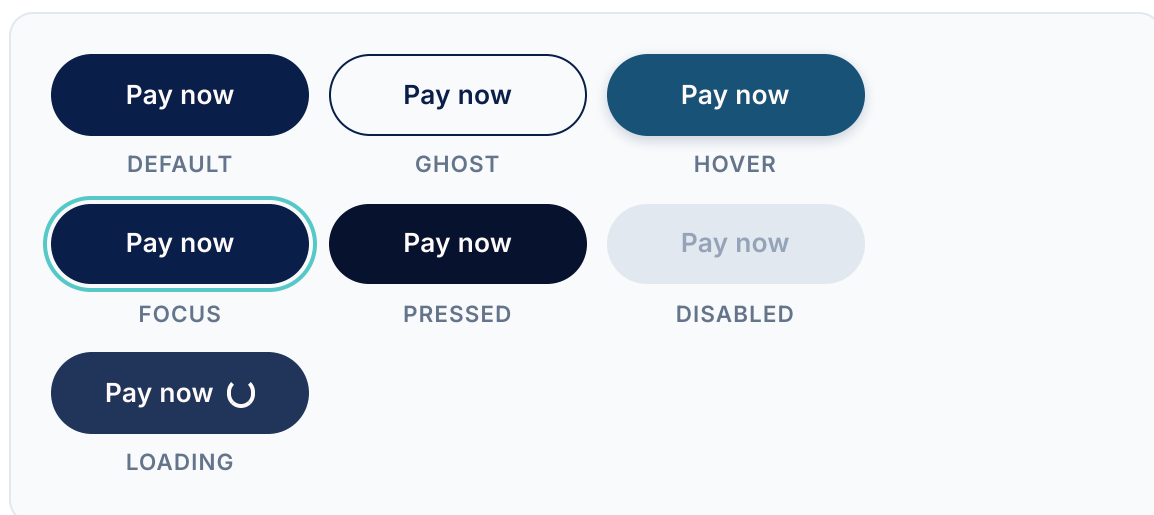
→ **Split the prototype into four independent flows (see Prototype).**

Week 6 · Submission

Figma shares the entire design page, and the submission portal rejected a URL text file.

→ **Packaged a view-only hyperlink inside a PDF for reviewer access.**

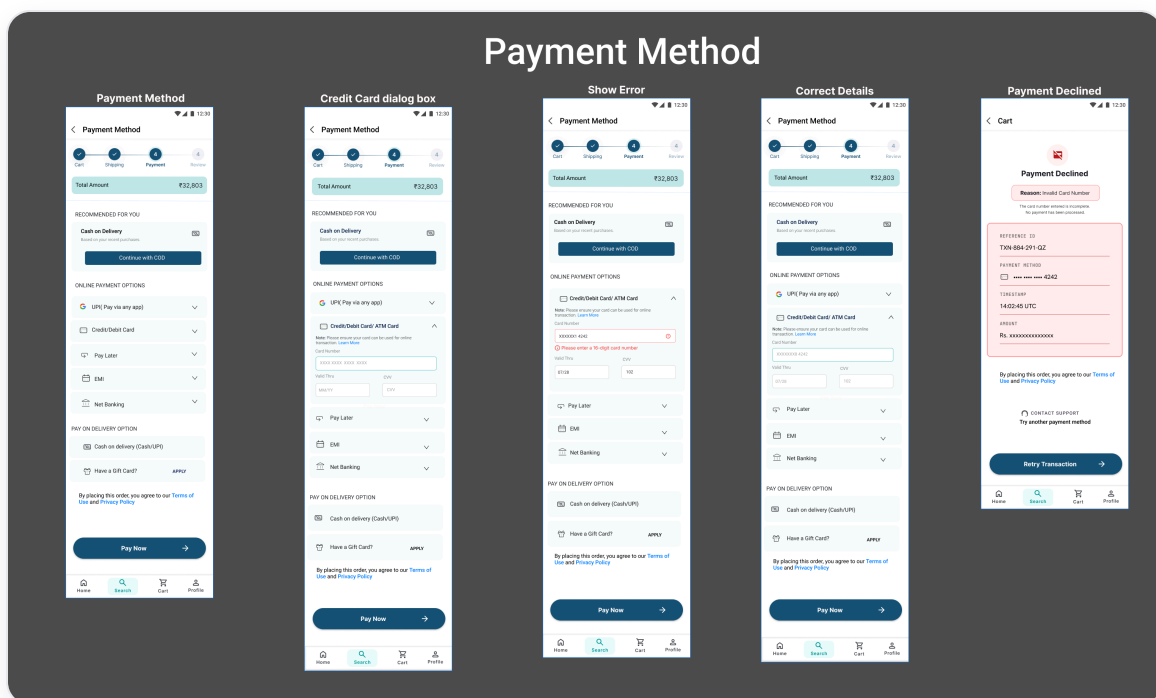
Example — button states. Because a static frame can't show real hover/focus/pressed behaviour, each interaction state was designed as an explicit pill-button variant in the app palette:



04 Solution

A redesigned checkout anchored on three principles drawn directly from the research: **clarity** (transparent totals, calm colors with a clear purpose per color), **recovery** (every error state leads somewhere, never to a dead end), and **accessibility** (WCAG 2.2 AA contrast and a documented keyboard tab order from the start).

The final payment screen consolidates method selection, a recommendation nudge ("In your previous order, you used COD"), and a single confident primary action — Pay Now.



Final payment-method screen — method selection with contextual recommendation and a single primary action.

05 Research

Research ran across all eight weeks and combined three methods: heuristic teardowns of live e-commerce checkouts (week 1), an accessibility contrast audit against WCAG 2.2 AA (week 5), and a structured A/B test proposal for guest-checkout visibility (week 7).

Heuristic teardowns

Studied navigation menus, search, filters, buttons, forms, product cards, carts, and checkout screens to identify usability issues. Learning that *every element has a reason for its placement* — nothing is random — reshaped how I approached hierarchy and information scent.

Pricing transparency

Investigated how MRP, taxes, shipping, discounts, and platform fees combine — and the roles of sellers, brands, and platforms. Confirmed that perceived fairness of the final price outweighs speed and polish.

A/B test proposal — guest checkout visibility

Hypothesis. Making *Continue as Guest* more visible and showing a guest-checkout notification during checkout will reduce user confusion, improve user confidence, and increase successful guest-checkout completion.

VERSION A · CONTROL

Current design

- Continue as Guest appears *below* the Sign In option.
- No guest-checkout notification is displayed.
- Users proceed without confirmation they're checking out as a guest.

VERSION B · VARIANT

Improved design

- Display **Continue as Guest** as the primary action.
- Surface this notification on the Shipping Address screen after Guest Checkout is selected:

You are currently checking out as a guest. Sign in at

any time to save your details.

Metrics

⦿ Time taken to locate **Continue as Guest**

⦿ % of users selecting **Continue as Guest**

⦿ Number of users asking whether sign-in is required

⦿ % who notice and understand the guest-checkout notification

⦿ Task completion rate

⦿ User confidence during checkout (participant feedback)

Risks

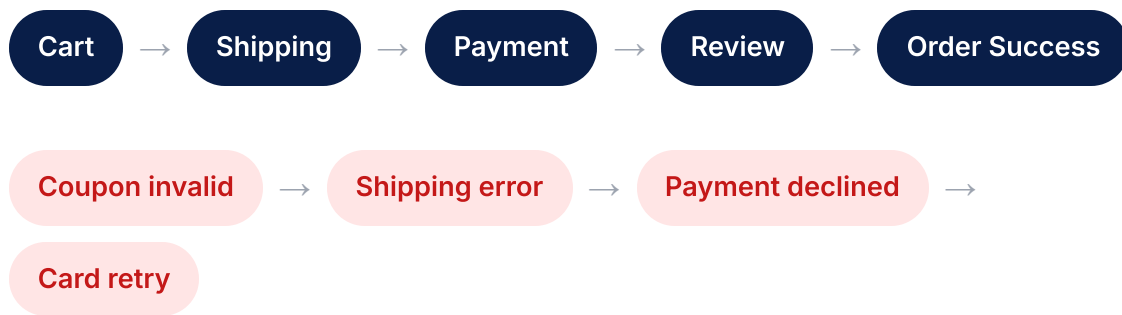
- Returning users may overlook the **Sign In** option.
- Users may ignore the guest-checkout notification.
- A more prominent Guest option may reduce account sign-ins.
- The notification may add visual clutter if too prominent.

Expected outcome. Users find the Guest Checkout option more easily, understand an account isn't required, and complete checkout with greater confidence and less confusion.

06 User Flow

The checkout was mapped as a happy path plus four explicit error branches, because a single linear prototype couldn't represent the real range of outcomes. Designing recovery paths for declined payments, invalid

coupons, and shipping errors forced clarity on *what happens when things go wrong*.



Each branch was built as its own prototype flow so reviewers could experience the recovery journey in isolation, rather than relying on a single happy-path demo.

07 Information Architecture

Because the checkout couldn't be one linear prototype, it was architected as a root with four independent flows. Each flow owns a specific slice of the journey — the happy path end-to-end, plus three error/recovery branches — so every scenario can be reviewed in isolation.

Checkout

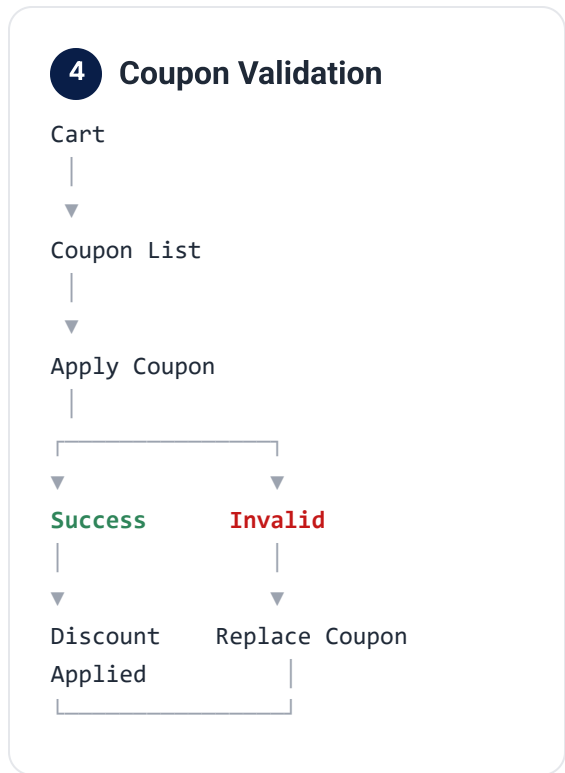
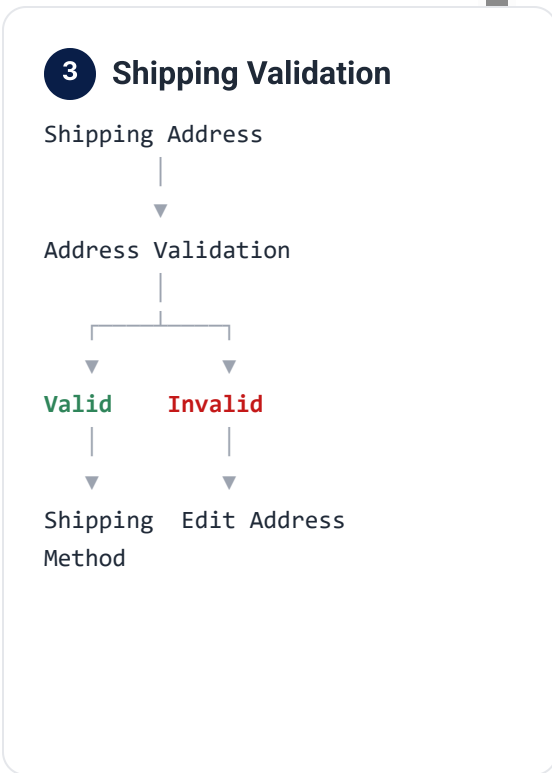
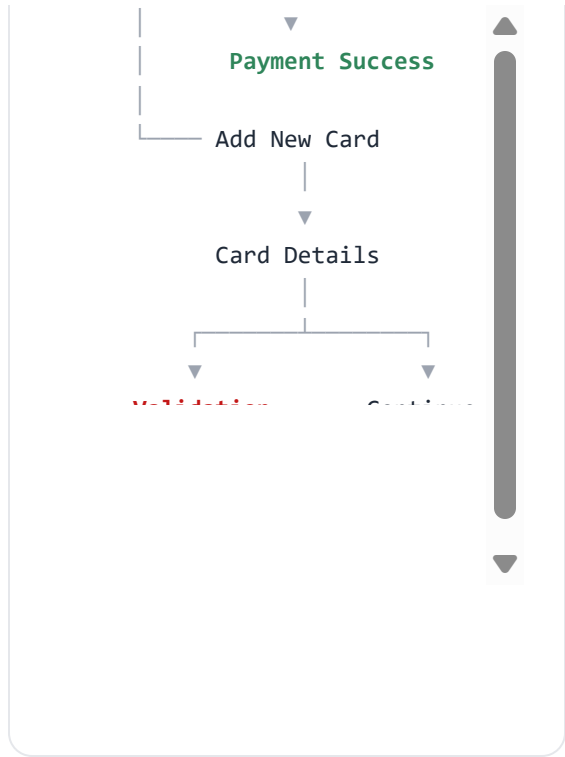
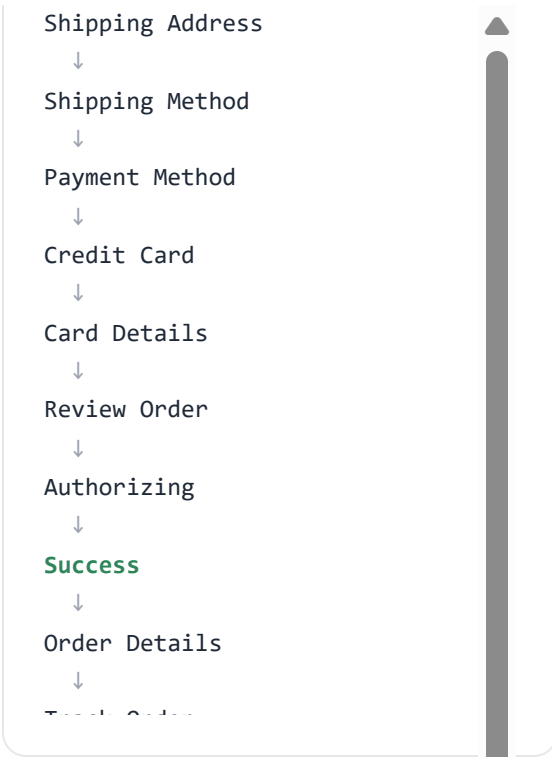
- |
- |— Flow 01 – Happy Path
- |— Flow 02 – Payment Method
- |— Flow 03 – Shipping Validation
- |— Flow 04 – Coupon Validation

1 Happy Path

Cart
↓
Login
↓

2 Payment Method

Payment Method
|
|— Saved Card
|



08 Wireframes

Early structure focused on what information belongs on each screen and in what order — not visuals. The cart wireframe alone surfaced several structural decisions that carried into hi-fi: collapse the cart summary, reduce nav icons from 5 to 4, and make the Proceed button large with a "remaining steps" hint.

Key wireframe decisions

- Progress stepper visible on every step
- Order summary collapsible, not always-expanded
- Out-of-stock items grouped and visually muted
- Primary action fixed and large at the bottom

Refinements before hi-fi

- Removed Wishlist icon from bottom nav (5 → 4)
- Shrunk delete icon 15px → 13px
- Dimmed "Save for Later" to 70% opacity, 16px → 14px
- Applied coupon: gradient fill on "Applied" badge

09 Design System

Before hi-fi, I defined the design tokens that would govern the whole flow — color, typography, spacing, and radius — captured as a structured `tokens.json` managed in VS Code and version-controlled on GitHub. The palette was chosen for brand identity, accessibility, visual hierarchy, and eye comfort (the prior design had too many colors and caused eye strain).

→ [See the full visual design system](#)

Color tokens

- Primary #091E48 — brand navy
- Secondary #185277 — supporting navy
- Tertiary #56C8C8 — teal accent, focus rings

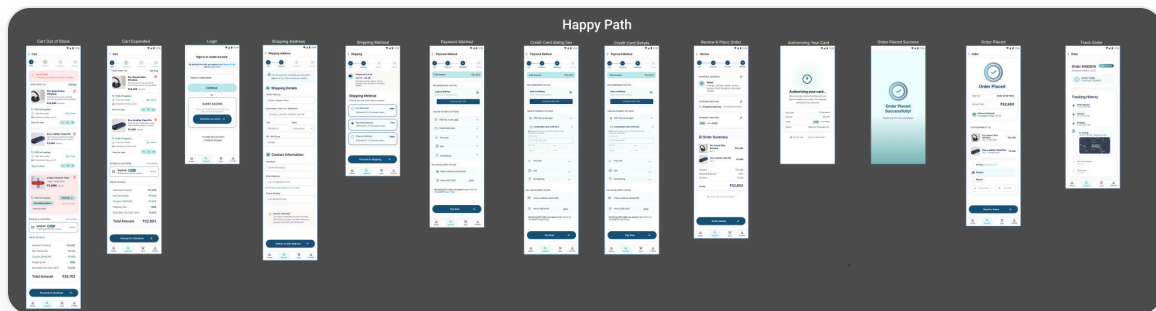
Spacing & radius tokens

- Spacing: 4 / 8 / 12 / 16 / 24 px
- Radius: 4px (small) / 8px (large)
- Type pairing: Roboto (headings) + Inter (body)

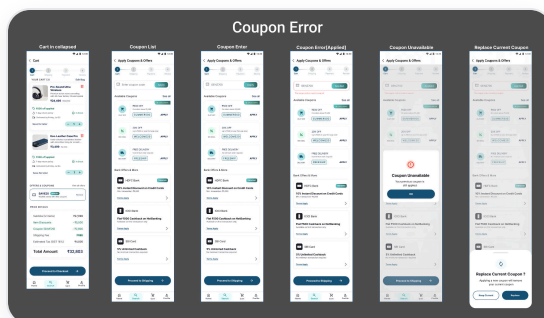
- Error #FF3333 · Success #2F855A
- Each color given an explicit, named purpose

10 High-Fidelity UI

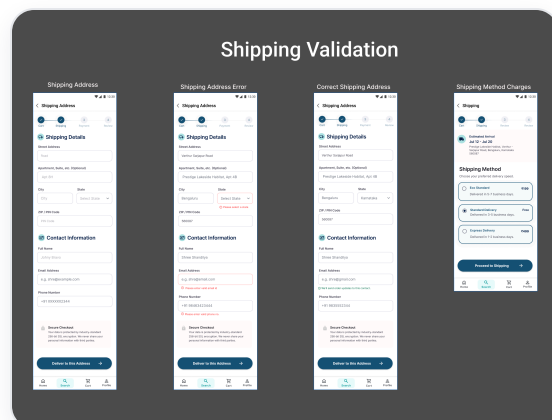
Hi-fi screens applied the token system to every component — buttons, text fields (with all six states), cards, progress stepper, and badges. Beyond cosmetics, the hi-fi pass resolved the interaction details Figma couldn't simulate: documented tab order, validation timing, and the exact gradient on success/confirmation surfaces.



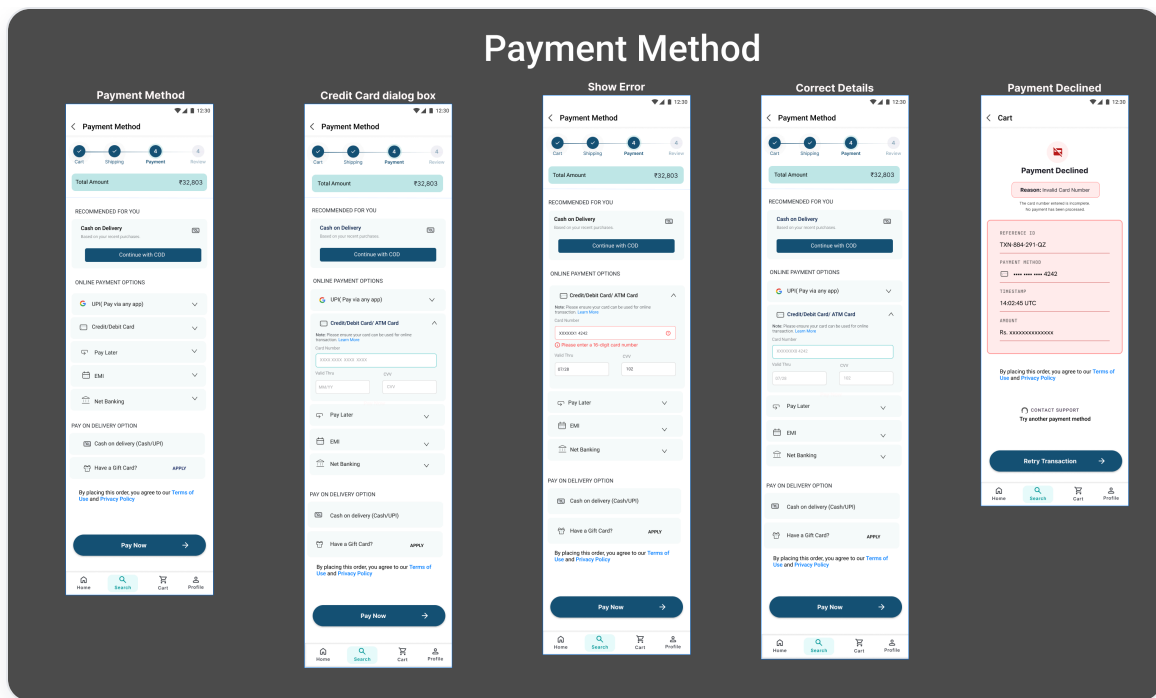
Happy-path flow — cart through order success, applying the token system end to end.



Coupon error — inline validation with a clear recovery path.



Shipping validation — error messaging tied to a specific field.

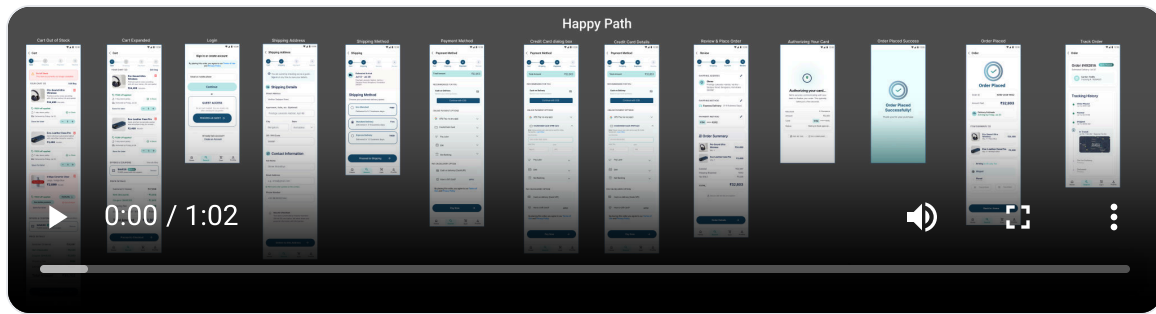


Payment method — method selection with contextual recommendation and a single primary action.

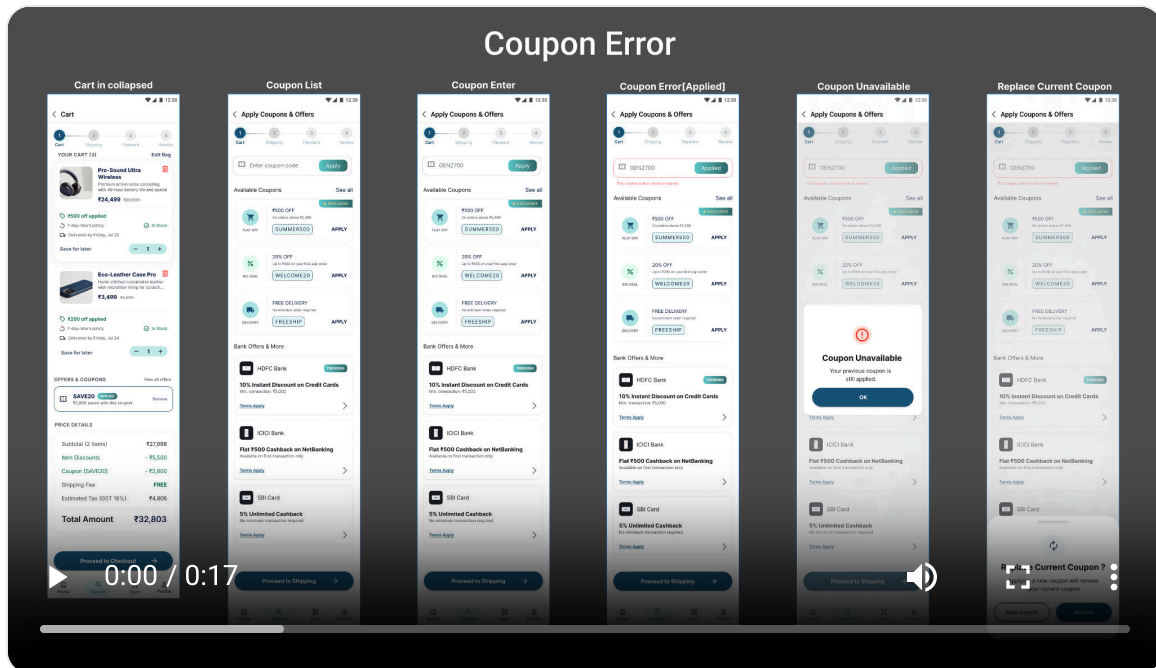
11 Prototype

The interactive prototype was split into four independent flows — Happy Path, Coupon Edge Cases, Payment Failure, and Shipping Validation — because Figma's single-interaction model couldn't branch a Pay Now button to either success or failure based on input. Interactive component variants plus Smart Animate carried the micro-interactions that real text entry couldn't.

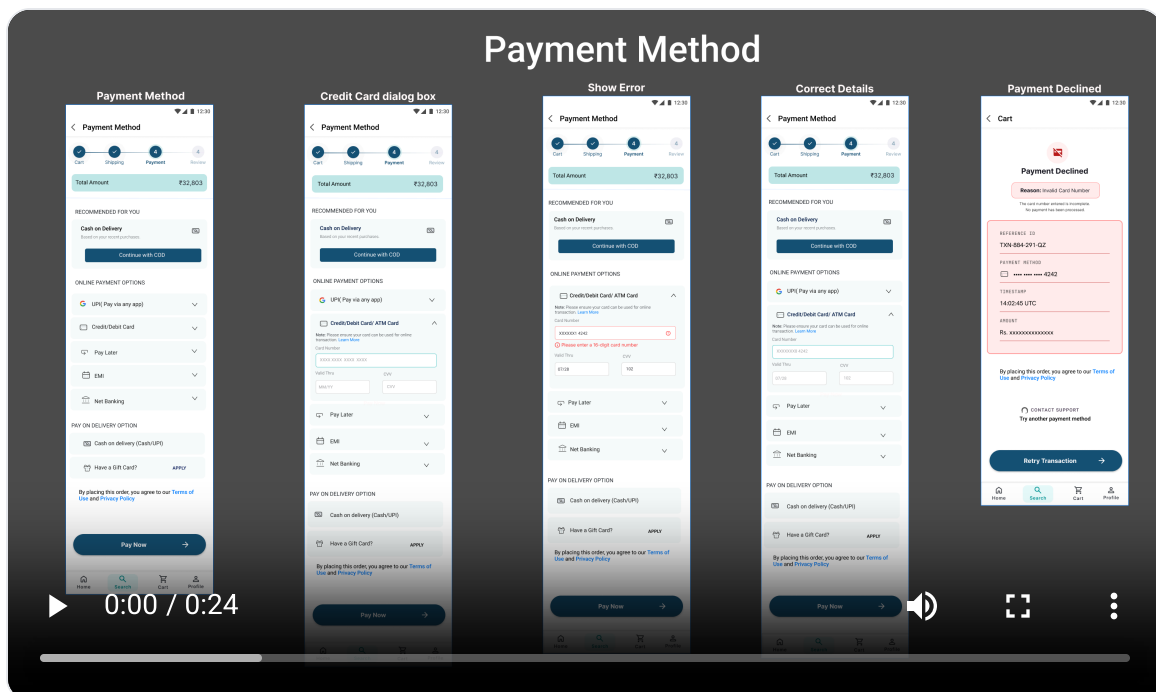
Trade-off. Four separate flows are less elegant than one continuous demo, but they let each error-recovery journey be experienced clearly and on its own terms — a better test of the design's resilience than a happy-path-only walkthrough.



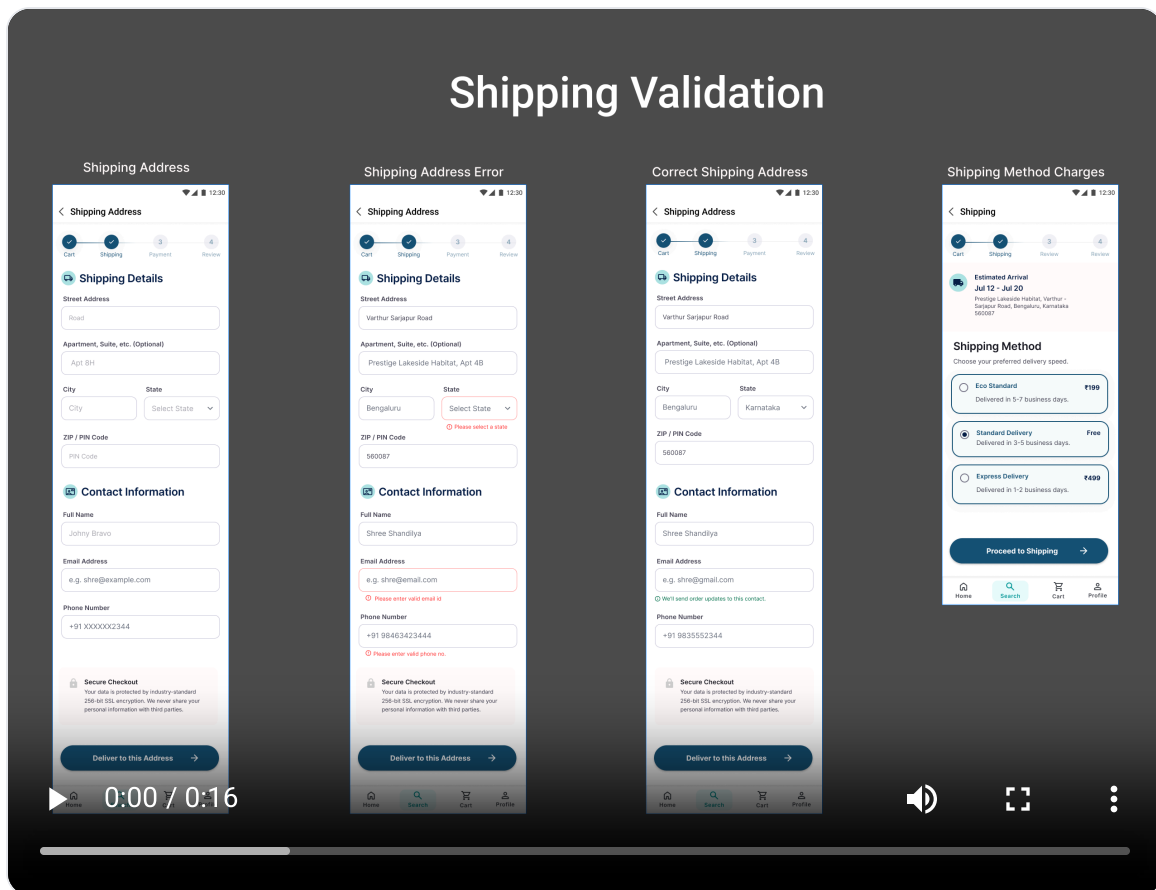
1 Happy Path



2 Coupon Edge Case



3 Payment Failure & Recovery



4 Shipping Validation

12 Testing

Testing had two pillars: an accessibility contrast audit against WCAG 2.2 AA, and documented keyboard focus order for the screens Figma couldn't navigate by keyboard. Both surfaced concrete, fixable issues — not vague "make it better" feedback.

WCAG 2.2 AA contrast audit

Every text/UI pair on the checkout screens was measured against WCAG 2.2 AA ($\geq 4.5:1$). Of 18 checks, 16 pass and 2 are flagged for review (non-interactive surfaces, exempt under WCAG). The text-field contrast was corrected from a near-miss to a clear pass.

SCREEN	ELEMENT	FOREGROUND	BACKGROUND	RATIO
Checkout	Page Title	 #000000	 #FFFFFF	21:1
Checkout	Product Name	 #295174	 #DFF1F2	7.13
Checkout	Body Text	 #0E1E45	 #FFFFFF	16.2
Checkout	Primary Button Text	 #FFFFFF	 #295174	8.32
Checkout	Secondary Button Text	 #56617D	 #FFFFFF	6.17
Checkout	Input Border	 #244664	 #FFFFFF	9.83
Checkout	Error Text	 #DC2626	 #FFFFFF	4.83
Checkout	Success Text	 #2D6A4C	 #FFFFFF	6.4:
Checkout	Progress Stepper	 #295174	 #DFF1F2	7.13
Checkout	Disabled Text	 #464554	 #FFFFFF	9.37
Checkout	Out of stock label	 #F1BF41	 #FCEEEED	1.51
Checkout	Out of stock label	 #9D5E0B	 #FCEEEED	4.6:
Coupon	Coupon code	 #0E1E45	 #E5F4F5	14.4
Coupon	Icon	 #0E1E45	 #B6E1E2	11.5:
Coupon	Secure Check	 #1B1B23	 #FEFAFA	16.5:
Coupon	Order successful text	 #295174	 #F0FEFF	8.05
Invalid Payment	Card of payment opt.	 #EB483E	 #F9FDFD	3.7:
Payment Declined	Card tab error	 #1B1B23	 #FCEFEF	16.5:

Keyboard tab order

Because Figma can't render real Tab-order focus traversal, the intended focus order for the two payment-error screens was documented as numbered sequences — ready for HTML/CSS/JS implementation under WCAG 2.4.7 (Focus Order).

Invalid Payment — 15 steps

- 1 Back Button
- 2 Progress Stepper
- 3 Recommend for You – Payment Method: Cash on Delivery (COD)
- 4 Payment Method – UPI
- 5 Credit/Debit Card / ATM Card
- 6 Card Number Field
- 7 Expiry Date Field
- 8 CVV Field
- 9 Pay Later
- 10 EMI
- 11 Net Banking
- 12 Cash on Delivery (Cash/UPI)
- 13 Have a Gift Card?
- 14 Terms & Conditions Checkbox
- 15 Pay Now Button

Declined Payment — 5 steps

- 1 Back Button
- 2 Terms & Conditions Checkbox
- 3 Contact Support
- 4 Try Another Payment Method — navigates the user to alternative payment options such as UPI, Card, Wallet, or Cash on Delivery
- 5 Retry Transaction

Why it matters.

Documenting focus order turns "the design is accessible" from a claim into a spec the developer can implement and QA can verify — closing the gap Figma leaves open.

13 Outcomes

The redesigned checkout delivered a calmer, more resilient, and measurably more accessible flow — and a set of documented decisions ready for frontend handoff.

- ✓ A token-based design system that replaced an eye-straining multi-color palette with a purposeful navy/teal scheme.

- ✓ Four explicit error-recovery flows, so no checkout failure ends in a dead end.
- ✓ Every audited contrast pair passing WCAG 2.2 AA, with documented keyboard tab order for accessibility handoff.
- ✓ A structured A/B test proposal with hypothesis, metrics, and risks ready to run on guest-checkout visibility.
- ✓ Per-screen change log (cart, out-of-stock, payment, success, track-order) capturing every iteration decision.

14 What I Learned

Three lessons stood out across the eight weeks — about sequencing, tool limits, and what "done" really means in UX.

System before screens

If I redid it, I'd build the design system — typography, spacing, reusable components — *before* the screens. Planning tokens first would have kept the prototype far more consistent and saved rework.

Plan edge cases early

Mapping all user flows and edge cases up front (coupon, shipping, payment, declined card) makes the prototype far more organized than discovering branches mid-build.

Design for the tool's limits

Figma can't do conditional logic, live inputs, or real keyboard nav. Documenting intent (tab order, state variants, separate flows) is part of the deliverable — not a workaround to hide.

UX is more than UI

I was surprised how much planning sits beyond the screens: usability testing, accessibility checks, spacing consistency, A/B testing, and documenting limitations are all essential to a user-centered result.

Next step: code it

I'd explore converting the Figma prototype into HTML/CSS to implement the interactions Figma can't — keyboard focus, live validation, and true branching.

Colors need a purpose

Each color should have a defined purpose and naming convention. The previous design's eye strain came from too many colors used without intent.

Explore the rest

Browse the design tokens and components behind this case study.

[Design System](#)[Design Guidelines](#)